01 **EXPERIMENT 7 | VERILOG** Part 1/1

Basic Gates — (A) AND gate

and.v

```
// Gate level
module and_gl (input A, B, output Y);
  and (Y, A, B);
endmodule
// Data flow
module and_df (input A, B, output Y);
  assign Y = A & B;
endmodule
// Behavioural
module and_bh (input A, B, output reg Y);
  always @(*) begin
    if (A == 1 && B == 1)
      Y = 1;
    else
      Y = 0;
  end
endmodule
```

and_tb.v

```
// test bench
module and_tb;
  reg A, B;
  wire Y_gl;
  wire Y_df;
  wire Y_bh;
  and_gl testa (A, B, Y_gl);
  and_df testb (A, B, Y_df);
  and_bh testc (A, B, Y_bh);
  initial begin
    A = 0; B = 0;
    #1 B = 1;
    #1 A = 1; B = 0;
    #1 B = 1;
  end
  initial begin
    $monitor("%t | A = %b | B = %b | Y_gl = %b | Y_df = %b | Y_bh = %b", $time, A,
         B, Y_gl, Y_df, Y_bh);
    $dumpfile("dump_and.vcd");
    $dumpvars();
  end
endmodule
```

02 **EXPERIMENT 7 | VERILOG** **Part 1/2 (continues →)**

Basic Gates — (B) OR gate

or.v

```
// Gate level
module or_gl (input A, B, output Y);
  or (Y, A, B);
endmodule
// Data flow
module or_df (input A, B, output Y);
  assign Y = A | B;
endmodule
// Behavioural
module or_bh (input A, B, output reg Y);
  always @(*) begin
    if (A == 1 || B == 1)
       Y = 1;
     else
       Y = 0;
  end
endmodule
```

03 **EXPERIMENT 7 | VERILOG** **Part 2/2**

Basic Gates — (B) OR gate

or_tb.v

```
// test bench
module or_tb;
  reg A, B;
  wire Y_gl;
  wire Y_df;
  wire Y_bh;
  or_gl testa (A, B, Y_gl);
  or_df testb (A, B, Y_df);
  or_bh testc (A, B, Y_bh);
  initial begin
    A = 0; B = 0;
    #1 B = 1;
    #1 A = 1; B = 0;
    #1 B = 1;
  end
  initial begin
    $monitor("%t | A = %b | B = %b | Y_gl = %b | Y_df = %b | Y_bh = %b", $time, A,
        B, Y_gl, Y_df, Y_bh);
    $dumpfile("dump_or.vcd");
    $dumpvars();
  end
endmodule
```

04 **EXPERIMENT 7 | VERILOG** Part 1/1

Basic Gates — (C) NOT gate

not.v

```
// Gate level
module not_gl (input A, output Y);
  not (Y, A);
endmodule
// Data flow
module not_df (input A, output Y);
  assign Y = ~A;
endmodule
// Behavioural
module not_bh (input A, output reg Y);
  always @(*) begin
    if (A == 0)
      Y = 1;
    else
      Y = 0;
  end
endmodule
```

not_tb.v

```
// test bench
module not_tb;
  reg A;
  wire Y_gl;
  wire Y_df;
  wire Y_bh;
  not_gl testa (A, Y_gl);
  not_df testb (A, Y_df);
  not_bh testc (A, Y_bh);
  initial begin
    A = 0;
    #1 A = 1;
  end
  initial begin
    $monitor("%t | A = %b | Y_gl = %b | Y_df = %b | Y_bh = %b", $time, A, Y_gl,
         Y_df, Y_bh);
    $dumpfile("dump_not.vcd");
    $dumpvars();
  end
endmodule
```

05 EXPERIMENT 7 | VERILOG

Part 1/1

Basic Gates — (D) NAND gate

nand.v

```
// Gate level
module nand_gl (input A, B, output Y);
  nand (Y, A, B);
endmodule
// Data flow
module nand_df (input A, B, output Y);
  assign Y = ~(A & B);
endmodule
// Behavioural
module nand_bh (input A, B, output reg Y);
  always @(*) begin
    if (A == 1 && B == 1)
      Y = 0;
    else
      Y = 1;
  end
endmodule
```

nand_tb.v

```
// test bench
module nand_tb;
  reg A, B;
  wire Y_gl;
  wire Y_df;
  wire Y_bh;
  nand_gl testa (A, B, Y_gl);
  nand_df testb (A, B, Y_df);
  nand_bh testc (A, B, Y_bh);
  initial begin
    A = 0; B = 0;
    #1 B = 1;
    #1 A = 1; B = 0;
    #1 B = 1;
  end
  initial begin
    $monitor("%t | A = %b | B = %b | Y_gl = %b | Y_df = %b | Y_bh = %b", $time, A,
         B, Y_gl, Y_df, Y_bh);
    $dumpfile("dump_nand.vcd");
    $dumpvars();
  end
endmodule
```

06 EXPERIMENT 7 | VERILOG

Part 1/2 (continues →)

Basic Gates — (E) NOR gate

nor.v

```
// Gate level
module nor_gl (input A, B, output Y);
  nor (Y, A, B);
endmodule
// Data flow
module nor_df (input A, B, output Y);
  assign Y = ~(A | B);
endmodule
// Behavioural
module nor_bh (input A, B, output reg Y);
  always @(*) begin
    if (A == 0 && B == 0)
      Y = 1;
    else
      Y = 0;
  end
endmodule
```

07 EXPERIMENT 7 | VERILOG

Part 2/2

Basic Gates — (E) NOR gate

nor_tb.v

```
// test bench
module nor_tb;
  reg A, B;
  wire Y_gl;
  wire Y_df;
  wire Y_bh;
  nor_gl testa (A, B, Y_gl);
  nor_df testb (A, B, Y_df);
  nor_bh testc (A, B, Y_bh);
  initial begin
    A = 0; B = 0;
    #1 B = 1;
    #1 A = 1; B = 0;
    #1 B = 1;
  end
  initial begin
    $monitor("%t | A = %b | B = %b | Y_gl = %b | Y_df = %b | Y_bh = %b", $time, A,
         B, Y_gl, Y_df, Y_bh);
    $dumpfile("dump_nor.vcd");
    $dumpvars();
  end
endmodule
```

08 EXPERIMENT 7 | VERILOG

Part 1/1

Basic Gates — (F) XOR gate

xor.v

```
// Gate level
module xor_gl (input A, B, output Y);
  xor (Y, A, B);
endmodule
// Data flow
module xor_df (input A, B, output Y);
  assign Y = A ^ B;
endmodule
// Behavioural
module xor_bh (input A, B, output reg Y);
  always @(*) begin
    if (A != B)
      Y = 1;
    else
      Y = 0;
  end
endmodule
```

xor_tb.v

```
// test bench
module xor_tb;
  reg A, B;
  wire Y_gl;
  wire Y_df;
  wire Y_bh;
  xor_gl testa (A, B, Y_gl);
  xor_df testb (A, B, Y_df);
  xor_bh testc (A, B, Y_bh);
  initial begin
    A = 0; B = 0;
    #1 B = 1;
    #1 A = 1; B = 0;
    #1 B = 1;
  end
  initial begin
    $monitor("%t | A = %b | B = %b | Y_gl = %b | Y_df = %b | Y_bh = %b", $time, A,
         B, Y_gl, Y_df, Y_bh);
    $dumpfile("dump_xor.vcd");
    $dumpvars();
  end
endmodule
```

09 **EXPERIMENT 8 | VERILOG** Part 1/1

Combinational Circuits — Half adder

half_adder.v

```
// Data flow
module half_adder_df (input A, B, output Sum, Cout);
  assign Sum = A ^ B;
  assign Cout = A & B;
endmodule
// Behavioural
module half_adder_bh (input A, B, output reg Sum, Cout);
  always @(*) begin
    if (A ^ B)
      Sum = 1;
    else
      Sum = 0;
    if (A & B)
      Cout = 1;
    else
      Cout = 0;
  end
endmodule
```

half_adder_tb.v

```
// test bench
module half_adder_tb;
  reg A, B;
  wire Sum_df;
  wire Cout_df;
  wire Sum_bh;
  wire Cout_bh;
  half_adder_df testa (A, B, Sum_df, Cout_df);
  half_adder_bh testb (A, B, Sum_bh, Cout_bh);
  initial begin
    {A, B} = 2'b00;
    #1 {A, B} = 2'b01;
    #1 {A, B} = 2'b10;
    #1 {A, B} = 2'b11;
  end
  initial begin
    $monitor("%t | A = %b | B = %b | df: Sum = %b Cout = %b | bh: Sum = %b Cout = %b", $time, A, B,
        Sum_df, Cout_df, Sum_bh, Cout_bh);
    $dumpfile("dump_half_adder.vcd");
    $dumpvars();
  end
endmodule
```

10 **EXPERIMENT 8 | VERILOG** **Part 1/2 (continues →)**

Combinational Circuits — (A) Half subtractor

half_subtractor.v

```
// Data flow
module half_subtractor_df (input A, B, output Diff, Borrow);
  assign Diff = A ^ B;
  assign Borrow = ~A & B;
endmodule
// Behavioural
module half_subtractor_bh (input A, B, output reg Diff, Borrow);
  always @(*) begin
    if (A ^ B)
      Diff = 1;
    else
      Diff = 0;
    if (~A & B)
      Borrow = 1;
    else
      Borrow = 0;
  end
endmodule
```

11 **EXPERIMENT 8 | VERILOG** **Part 2/2**

Combinational Circuits — (A) Half subtractor

half_subtractor_tb.v

```
// test bench
module half_subtractor_tb;
  reg A, B;
  wire Diff_df;
  wire Borrow_df;
  wire Diff_bh;
  wire Borrow_bh;
  half_subtractor_df testa (A, B, Diff_df, Borrow_df);
  half_subtractor_bh testb (A, B, Diff_bh, Borrow_bh);
  initial begin
    {A, B} = 2'b00;
    #1 {A, B} = 2'b01;
    #1 {A, B} = 2'b10;
    #1 {A, B} = 2'b11;
  end
  initial begin
    $monitor("%t | A = %b | B = %b | df: Diff = %b Borrow = %b | bh: Diff = %b Borrow = %b", $time,
        A, B, Diff_df, Borrow_df, Diff_bh, Borrow_bh);
    $dumpfile("dump_half_subtractor.vcd");
    $dumpvars();
  end
endmodule
```

12 **EXPERIMENT 8 | VERILOG** Part 1/1

Combinational Circuits — (B) Full adder

full_adder.v

```
// Data flow
module full_adder_df (input A, B, Cin, output Sum, Cout);
  assign Sum = A ^ B ^ Cin;
  assign Cout = (A & B) | (B & Cin) | (A & Cin);
endmodule
// Behavioural
module full_adder_bh (input A, B, Cin, output reg Sum, Cout);
  always @(*) begin
    if (A ^ B ^ Cin)
      Sum = 1;
    else
      Sum = 0;
    if ((A & B) | (B & Cin) | (A & Cin))
      Cout = 1;
    else
      Cout = 0;
  end
endmodule
```

full_adder_tb.v

```
// test bench
module full_adder_tb;
  reg A, B, Cin;
  wire Sum_df;
  wire Cout_df;
  wire Sum_bh;
  wire Cout_bh;
  full_adder_df testa (A, B, Cin, Sum_df, Cout_df);
  full_adder_bh testb (A, B, Cin, Sum_bh, Cout_bh);
  initial begin
    {A, B, Cin} = 3'b000;
    #1 {A, B, Cin} = 3'b001;
    #1 {A, B, Cin} = 3'b010;
    #1 {A, B, Cin} = 3'b011;
    #1 {A, B, Cin} = 3'b100;
    #1 {A, B, Cin} = 3'b101;
    #1 {A, B, Cin} = 3'b110;
    #1 {A, B, Cin} = 3'b111;
  end
  initial begin
    $monitor("%t | A = %b | B = %b | Cin = %b | df: Sum = %b Cout = %b | bh: Sum = %b Cout = %b",
        $time, A, B, Cin, Sum_df, Cout_df, Sum_bh, Cout_bh);
    $dumpfile("dump_full_adder.vcd");
    $dumpvars();
  end
endmodule
```

13 EXPERIMENT 8 | VERILOG

Part 1/1

Combinational Circuits — (C) Full subtractor

full_subtractor.v

```
// Data flow
module full_subtractor_df (input A, B, Bin, output Diff, Borrow);
  assign Diff = A ^ B ^ Bin;
  assign Borrow = (~A & B) | ((~A | B) & Bin);
endmodule
// Behavioural
module full_subtractor_bh (input A, B, Bin, output reg Diff, Borrow);
  always @(*) begin
    if (A ^ B ^ Bin)
      Diff = 1;
    else
      Diff = 0;
    if ((~A & B) | ((~A | B) & Bin))
      Borrow = 1;
    else
      Borrow = 0;
  end
endmodule
```

full_subtractor_tb.v

```
// test bench
module full_subtractor_tb;
  reg A, B, Bin;
  wire Diff_df;
  wire Borrow_df;
  wire Diff_bh;
  wire Borrow_bh;
  full_subtractor_df testa (A, B, Bin, Diff_df, Borrow_df);
  full_subtractor_bh testb (A, B, Bin, Diff_bh, Borrow_bh);
  initial begin
    {A, B, Bin} = 3'b000;
    #1 {A, B, Bin} = 3'b001;
    #1 {A, B, Bin} = 3'b010;
    #1 {A, B, Bin} = 3'b011;
    #1 {A, B, Bin} = 3'b100;
    #1 {A, B, Bin} = 3'b101;
    #1 {A, B, Bin} = 3'b110;
    #1 {A, B, Bin} = 3'b111;
  end
  initial begin
    $monitor("%t | A = %b | B = %b | Bin = %b | df: Diff = %b Borrow = %b | bh: Diff = %b Borrow =
        %b", $time, A, B, Bin, Diff_df, Borrow_df, Diff_bh, Borrow_bh);
    $dumpfile("dump_full_subtractor.vcd");
    $dumpvars();
  end
endmodule
```

14 EXPERIMENT 8 | VERILOG

Part 1/2 (continues →)

Combinational Circuits — (D) 4:1 multiplexer

mux.v

```
// Data flow
module mux_dataflow (Z, D, S);
  input [3:0] D;
  input [1:0] S;
  output Z;
  assign Z = (~S[1] & ~S[0] & D[0]) | (~S[1] & S[0] & D[1]) | (S[1] & ~S[0] & D[2]) |
      (S[1] & S[0] & D[3]);
endmodule
```

15 **EXPERIMENT 8 | VERILOG** **Part 2/2**

Combinational Circuits — (D) 4:1 multiplexer

```
// Behavioural
module mux_behavioural (Z, D, S);
  input [3:0] D;
  input [1:0] S;
  output reg Z;
  always @(*) begin
    case (S[1])
      0: begin
        if (S[0] == 0)
          Z = D[0];
        else
          Z = D[1];
      end
      1: begin
        if (S[0] == 0)
          Z = D[2];
        else
          Z = D[3];
      end
    endcase
  end
endmodule
```

mux_tb.v

```
// test bench
module mux_tb;
  reg [3:0] D;
  reg [1:0] S;
  wire Z_dataflow;
  wire Z_behavioural;
  mux_dataflow testa (Z_dataflow, D, S);
  mux_behavioural testb (Z_behavioural, D, S);
  initial begin
    D = 4'b0000;
    S = 2'b00;
    #1 S = 2'b01;
    #1 S = 2'b10;
    #1 S = 2'b11;
    #1 D = 4'b0001;
    #1 S = 2'b00;
    #1 D = 4'b1010; S = 2'b10;
  end
  initial begin
    $monitor("%t | D = %b | S = %b | Z_dataflow = %b | Z_behavioural = %b", $time,
         D, S, Z_dataflow, Z_behavioural);
    $dumpfile("dump1.vcd");
    $dumpvars();
  end
endmodule
```

16 **EXPERIMENT 8 | VERILOG** **Part 1/2 (continues →)**

Combinational Circuits — (E) 1:4 demultiplexer

demux.v

```
// Data flow
module demux_dataflow (Z, D, S);
  input [1:0] S;
  input D;
  output [3:0] Z;
  assign Z[0] = D & (~S[0]) & (~S[1]);
  assign Z[1] = D & (~S[1]) & (S[0]);
  assign Z[2] = D & (S[1]) & (~S[0]);
  assign Z[3] = D & (S[1]) & (S[0]);
endmodule
```

17 **EXPERIMENT 8 | VERILOG** **Part 2/2**

Combinational Circuits — (E) 1:4 demultiplexer

```
// Behavioural
module demux_behavioural (Z, D, S);
  input [1:0] S;
  input D;
  output reg [3:0] Z;
  always @(*) begin
    Z[0] = (S[1] == 0 && S[0] == 0) ? D : 0;
    Z[1] = (S[1] == 0 && S[0] == 1) ? D : 0;
    Z[2] = (S[1] == 1 && S[0] == 0) ? D : 0;
    Z[3] = (S[1] == 1 && S[0] == 1) ? D : 0;
  end
endmodule
```

demux_tb.v

```
// test bench
module demux_tb;
  reg D;
  reg [1:0] S;
  wire [3:0] Z_dataflow;
  wire [3:0] Z_behavioural;
  demux_dataflow testa (Z_dataflow, D, S);
  demux_behavioural testb (Z_behavioural, D, S);
  initial begin
    D = 1; S = 2'b00;
    #1 S = 2'b01;
    #1 S = 2'b10;
    #1 S = 2'b11;
  end
  initial begin
    $monitor("%t | D = %b | S1 = %b | S0 = %b | Z_dataflow = %b | Z_behavioural = %b", $time, D,
        S[1], S[0], Z_dataflow, Z_behavioural);
    $dumpfile("dump2.vcd");
    $dumpvars();
  end
endmodule
```

18 **EXPERIMENT 8 | VERILOG** **Part 1/2 (continues →)**

Combinational Circuits — (F) 4 to 2 encoder

encoder.v

```
// Data flow
module encoder_dataflow (A, Y);
  input [3:0] Y;
  output [1:0] A;
  assign A[1] = Y[3] | Y[2];
  assign A[0] = Y[3] | Y[1];
endmodule
// Behavioural
module encoder_behavioural (A, Y);
  input [3:0] Y;
  output reg [1:0] A;
  always @(Y) begin
    A[1] = Y[3] ? 1 : (Y[2] ? 1 : 0);
    A[0] = Y[3] ? 1 : (Y[1] ? 1 : 0);
  end
endmodule
```

19 **EXPERIMENT 8 | VERILOG** **Part 2/2**

Combinational Circuits — (F) 4 to 2 encoder

encoder_tb.v

```
// test bench
module encoder_tb;
  reg [3:0] Y;
  wire [1:0] A;
  wire [1:0] B;
  encoder_dataflow testa (A, Y);
  encoder_behavioural testb (B, Y);
  initial begin
    Y = 4'b0001;
    #1 Y = 4'b0010;
    #1 Y = 4'b0100;
    #1 Y = 4'b1000;
  end
  initial begin
    $monitor("%t | Y = %b | A = %b | B = %b", $time, Y, A, B);
    $dumpfile("dump3.vcd");
    $dumpvars();
  end
endmodule
```

20 **EXPERIMENT 8 | VERILOG** Part 1/1

Combinational Circuits — (G) 2 to 4 decoder

decoder.v

```
// Data flow
module decoder_dataflow (A, Y);
  input [1:0] A;
  output [3:0] Y;
  assign Y[0] = ~A[1] & ~A[0];
  assign Y[1] = ~A[1] & A[0];
  assign Y[2] = A[1] & ~A[0];
  assign Y[3] = A[1] & A[0];
endmodule
// Behavioural
module decoder_behavioural (A, Y);
  input [1:0] A;
  output reg [3:0] Y;
  always @(A) begin
    Y[0] = (A[1] == 0 && A[0] == 0) ? 1 : 0;
    Y[1] = (A[1] == 0 && A[0] == 1) ? 1 : 0;
    Y[2] = (A[1] == 1 && A[0] == 0) ? 1 : 0;
    Y[3] = (A[1] == 1 && A[0] == 1) ? 1 : 0;
  end
endmodule
```

decoder_tb.v

```
// test bench
module decoder_tb;
  reg [1:0] A;
  wire [3:0] Y;
  wire [3:0] Z;
  decoder_dataflow testa (A, Y);
  decoder_behavioural testb (A, Z);
  initial begin
    A = 2'b00;
    #1 A = 2'b01;
    #1 A = 2'b10;
    #1 A = 2'b11;
  end
  initial begin
    $monitor("%t | A = %b | Y_dataflow = %b | Y_behavioural = %b", $time, A, Y, Z);
    $dumpfile("dump4.vcd");
    $dumpvars();
  end
endmodule
```

21 **EXPERIMENT 8 | VERILOG** Part 1/1

Combinational Circuits — (H) 7 segment display decoder

seven_df.v

```
// Data flow
module seven (input [3:0] A, output a, b, c, d, e, f, g);
  assign a = (~A[3] & ~A[2] & ~A[1] & A[0]) | (~A[3] & A[2] & ~A[1] & ~A[0]) | (A[3] &
      ~A[2] & A[1] & A[0]) | (A[3] & ~A[2] & ~A[1] & A[0]);
  assign b = (~A[3] & A[2] & ~A[1] & A[0]) | (~A[3] & A[2] & A[1] & ~A[0]) | (A[3] &
      ~A[2] & A[1] & A[0]) | (A[3] & A[2] & ~A[1] & A[0]) | (~A[3] & ~A[2] & A[1] &
      A[0]);
  assign c = (~A[3] & ~A[2] & A[1] & ~A[0]) | (A[3] & ~A[2] & A[1] & ~A[0]) | (A[3] &
      A[2] & ~A[1] & ~A[0]) | (~A[3] & ~A[2] & A[1] & A[0]);
  assign d = (~A[3] & ~A[2] & ~A[1] & A[0]) | (~A[3] & A[2] & ~A[1] & ~A[0]) | (~A[3]
      & A[2] & A[1] & A[0]) | (A[3] & ~A[2] & ~A[1] & A[0]) | (A[3] & ~A[2] & A[1] &
      A[0]);
  assign e = (~A[3] & ~A[2] & ~A[1] & A[0]) | (~A[3] & ~A[2] & A[1] & A[0]) | (~A[3] &
      A[2] & ~A[1] & ~A[0]) | (~A[3] & A[2] & ~A[1] & A[0]);
  assign f = (~A[3] & ~A[2] & ~A[1] & A[0]) | (~A[3] & ~A[2] & A[1] & ~A[0]) | (~A[3]
      & A[2] & A[1] & A[0]) | (A[3] & ~A[2] & ~A[1] & A[0]);
  assign g = (~A[3] & ~A[2] & ~A[1] & ~A[0]) | (~A[3] & ~A[2] & ~A[1] & A[0]) | (~A[3]
      & A[2] & A[1] & A[0]) | (A[3] & ~A[2] & ~A[1] & A[0]) | (A[3] & ~A[2] & A[1] &
      A[0]);
endmodule
```

seven_bh.v

```
// Behavioural
module seven_bh (input [3:0] A, output reg a, b, c, d, e, f, g);
  always @(*) begin
    a = (A == 1 || A == 4) ? 0 : 1;
    b = (A == 5 || A == 6) ? 0 : 1;
    c = (A == 2) ? 0 : 1;
    d = (A == 1 || A == 4 || A == 7) ? 0 : 1;
    e = (A == 1 || A == 3 || A == 4 || A == 5 || A == 7 || A == 9) ? 0 : 1;
    f = (A == 1 || A == 2 || A == 3 || A == 7) ? 0 : 1;
    g = (A == 0 || A == 1 || A == 7) ? 0 : 1;
  end
endmodule
```

seven_tb.v

```
// test bench
module seven_tb;
  reg [3:0] A;
  wire a, b, c, d, e, f, g;
  seven test (A, a, b, c, d, e, f, g);
  initial begin
    for (A = 0; A <= 9; A = A + 1) begin
      #5;
    end
  end
  initial begin
    $monitor("%t, A=%b, a=%b, b=%b, c=%b, d=%b, e=%b, f=%b, g=%b", $time, A, a, b,
        c, d, e, f, g);
  end
endmodule
```

22 **EXPERIMENT 9 | VERILOG** Part 1/1

Flip-Flops — (A) SR latch

srlatch.v

```
module srlatch (input S, R, output reg Q, Qbar);
  always @(S or R) begin
    if (S == 0 && R == 0) begin
      Q = Q;
      Qbar = Qbar;
    end
    else if (S == 1 && R == 0) begin
      Q = 1;
      Qbar = 0;
    end
    else if (S == 0 && R == 1) begin
      Q = 0;
      Qbar = 1;
    end
    else begin
      Q = 1'bx;
      Qbar = 1'bx;
    end
  end
endmodule
```

srlatch_tb.v

```
// test bench
module srlatch_tb;
  reg S, R;
  wire Q, Qbar;
  srlatch test (S, R, Q, Qbar);
  initial begin
    S = 1; R = 0;
    #1 S = 0; R = 1;
    #1 S = 1; R = 0;
    #1 S = 0; R = 0;
    #1 S = 1; R = 1;
  end
  initial begin
    $monitor("%t | S = %b | R = %b | Q = %b | Qbar = %b", $time,
        S, R, Q, Qbar);
    $dumpfile("dump1.vcd");
    $dumpvars();
  end
endmodule
```

23 **EXPERIMENT 9 | VERILOG** Part 1/1

Flip-Flops — (B) D flip-flop

dff.v

```
module dff (input D, CLK, output reg Q, Qbar);
  always @(posedge CLK) begin
    Q = D;
    Qbar = ~D;
  end
endmodule
```

dff_tb.v

```
// test bench
module dff_tb;
  reg D, CLK;
  wire Q, Qbar;
  dff test (D, CLK, Q, Qbar);
  initial begin
    D = 1; CLK = 0;
    #1 D = 1; CLK = 1;
    #1 D = 1; CLK = 0;
    #1 D = 0; CLK = 1;
    #1 D = 1; CLK = 0;
    #1 D = 1; CLK = 1;
  end
  initial begin
    $monitor("%t | D = %b | CLK = %b | Q = %b | Qbar = %b", $time,
         D, CLK, Q, Qbar);
    $dumpfile("dump.vcd");
    $dumpvars();
  end
endmodule
```

24 **EXPERIMENT 9 | VERILOG** Part 1/1

Flip-Flops — (C) J-K flip-flop

jkff.v

```
module jkff (input J, K, CLK, output reg Q);
  always @(posedge CLK) begin
    case ({J, K})
       2'b00: begin Q = Q; end
       2'b01: begin Q = 0; end
       2'b10: begin Q = 1; end
       2'b11: begin Q = ~Q; end
    endcase
  end
endmodule
```

jkff_tb.v

```
// test bench
module jkff_tb;
  reg J, K, CLK;
  wire Q;
  jkff test (J, K, CLK, Q);
  initial CLK = 0;
  always #1 CLK = ~CLK;
  initial begin
    J = 0; K = 0;
    #1 J = 1; K = 0;
    #2 J = 0; K = 1;
    #2 J = 1; K = 0;
    #2 J = 1; K = 1;
    #2 J = 1; K = 1;
    $stop;
  end
  initial begin
    $monitor("%t | J = %b | K = %b | CLK = %b | Q = %b",
         $time, J, K, CLK, Q);
    $dumpfile("dump.vcd");
    $dumpvars();
  end
endmodule
```

25 **EXPERIMENT 9 | VERILOG** Part 1/1

Flip-Flops — (D) T flip-flop

tff.v

```
module tff (input T, CLK, output reg Q);
  initial Q = 0;
  always @(posedge CLK) begin
    case (T)
      0: Q = Q;
      1: Q = ~Q;
    endcase
  end
endmodule
```

tff_tb.v

```
// test bench
module tff_tb;
  reg T, CLK;
  wire Q;
  tff test (T, CLK, Q);
  initial CLK = 0;
  always #1 CLK = ~CLK;
  initial begin
    T = 0;
    #1 T = 1;
    #2 T = 0;
    #2 T = 1;
    #2 T = 1;
    #2 T = 0;
    $stop;
  end
  initial begin
    $monitor("%t | T = %b | CLK = %b | Q = %b", $time, T, CLK, Q);
    $dumpfile("dump.vcd");
    $dumpvars();
  end
endmodule
```

26 **EXPERIMENT 10 | VERILOG** **Part 1/2 (continues →)**

Asynchronous Counters And Shift Register — (A) Up counter

up_counter.v

```
// Up counter
module up_counter_tff_behavioural (input CLK, RESET, output [1:0] Q);
  wire [1:0] T;
  assign T[0] = 1;
  assign T[1] = Q[0];
  T_ff_behav T0 (.T(T[0]), .CLK(CLK), .RESET(RESET), .Q(Q[0]));
  T_ff_behav T1 (.T(T[1]), .CLK(CLK), .RESET(RESET), .Q(Q[1]));
endmodule
// T flip-flop
module T_ff_behav (input T, CLK, RESET, output reg Q);
  always @(posedge CLK or posedge RESET) begin
    if (RESET)
      Q <= 0;
    else if (T)
      Q <= ~Q;
    else
      Q <= Q;
  end
endmodule
```

27 **EXPERIMENT 10 | VERILOG** **Part 2/2**

Asynchronous Counters And Shift Register — (A) Up counter

up_counter_tb.v

```
// test bench
module up_counter_tff_behavioural_tb;
  reg CLK, RESET;
  wire [1:0] Q;
  up_counter_tff_behavioural test (.CLK(CLK), .RESET(RESET), .Q(Q));
  always #5 CLK = ~CLK;
  initial begin
    $monitor("Time = %t | RESET = %b | Count = %d | Binary = %b",
         $time, RESET, Q, Q);
    CLK = 0;
    RESET = 1;
    #10 RESET = 0;
    #100;
    RESET = 1;
    #10 RESET = 0;
    #50;
    $finish;
  end
endmodule
```

28 **EXPERIMENT 10 | VERILOG** Part 1/1

Asynchronous Counters And Shift Register — (B) Down counter

down_counter.v

```
// Down counter
module down_counter_tff_behavioural (input CLK, RESET, output [1:0] Q);
  wire [1:0] T;
  assign T[0] = 1;
  assign T[1] = ~Q[0];
  T_ff_behav T0 (.T(T[0]), .CLK(CLK), .RESET(RESET), .Q(Q[0]));
  T_ff_behav T1 (.T(T[1]), .CLK(CLK), .RESET(RESET), .Q(Q[1]));
endmodule
// T flip-flop (reset sets the output to 1)
module T_ff_behav (input T, CLK, RESET, output reg Q);
  always @(posedge CLK or posedge RESET) begin
    if (RESET)
      Q <= 1;
    else if (T)
      Q <= ~Q;
    else
      Q <= Q;
  end
endmodule
```

down_counter_tb.v

```
// test bench
module down_counter_tff_behavioural_tb;
  reg CLK, RESET;
  wire [1:0] Q;
  down_counter_tff_behavioural test (.CLK(CLK), .RESET(RESET), .Q(Q));
  always #5 CLK = ~CLK;
  initial begin
    $monitor("Time = %t | RESET = %b | Count = %d | Count = %b",
        $time, RESET, Q, Q);
    CLK = 0;
    RESET = 1;
    #10 RESET = 0;
    #100;
    RESET = 1;
    #10 RESET = 0;
    #5;
    $finish;
  end
endmodule
```

29 **EXPERIMENT 10 | VERILOG** Part 1/1

Asynchronous Counters And Shift Register — (C) SISO shift register

siso.v

```
// D flip-flop
module d_ff (input CLK, RESET, D, output reg Q);
  always @(posedge CLK or posedge RESET) begin
    if (RESET)
       Q <= 0;
    else
       Q <= D;
  end
endmodule
// SISO shift register
module siso_shift_register (input CLK, RESET, SERIAL_IN, output SERIAL_OUT);
  wire Q0, Q1, Q2, Q3;
  d_ff d0 (.CLK(CLK), .RESET(RESET), .D(SERIAL_IN), .Q(Q0));
  d_ff d1 (.CLK(CLK), .RESET(RESET), .D(Q0), .Q(Q1));
  d_ff d2 (.CLK(CLK), .RESET(RESET), .D(Q1), .Q(Q2));
  d_ff d3 (.CLK(CLK), .RESET(RESET), .D(Q2), .Q(Q3));
  assign SERIAL_OUT = Q3;
endmodule
```

siso_tb.v

```
// test bench
module tb_siso_shift_register();
  reg CLK, RESET, SERIAL_IN;
  wire SERIAL_OUT;
  siso_shift_register test (
       .CLK(CLK), .RESET(RESET),
       .SERIAL_IN(SERIAL_IN), .SERIAL_OUT(SERIAL_OUT)
       );
  initial begin
    CLK = 0;
    forever #5 CLK = ~CLK;
  end
  initial begin
    RESET = 1; SERIAL_IN = 0;
    #10 RESET = 0;
    SERIAL_IN = 1; #10;
    SERIAL_IN = 0; #10;
    SERIAL_IN = 1; #10;
    SERIAL_IN = 1; #10;
    SERIAL_IN = 0; #40;
    $finish;
  end
  initial begin
    $monitor("Time = %t | SERIAL_IN = %b | SERIAL_OUT = %b",
          $time, SERIAL_IN, SERIAL_OUT);
  end
endmodule
```